

Resolução dos Exercícios

Capítulo: Tipos de dados básicos

1. Indique quais das seguintes declarações estão corretas.

- ☐ a) `y int;`
- ☐ b) `int ;`
- ☐ c) `integer x;`
- ☐ d) `inta , b;`
- ☒ e) `float f, g , c;`
- ☐ f) `char ch1=ch2='A' ;`
- ☒ g) `char ch1 ='A', ch2='A';`

2. Uma variável inteira, quando é declarada, é sempre iniciada com...

- ☐ a) 0 (Zero)
- ☐ b) 1 (Um)
- ☒ c) **Um valor aleatório**
- ☐ d) Um valor negativo

3. Indique, na seguinte lista, quais os identificadores corretos de variáveis:

- ☒ a) **Valor**
- ☐ b) `&xvar`
- ☐ c) `dez%`
- ☐ d) `a+b`
- ☒ e) **`_Kabonga`**
- ☒ f) **`MENOS`**
- ☐ g) `10a`
- ☒ h) **`a10`**
- ☐ i) `main`

4. O nome de uma variável...

- ☒ a) Deve indicar aquilo que ela armazena
- ☐ b) Deve ser o menor possível
- ☐ c) Deve ser o maior possível
- ☒ d) Deve ser o mais explícito possível
- ☐ e) Deve ser todo escrito em maiúsculas
- ☒ f) Pode, mas não deve, começar por underscore (_)
- ☒ g) Pode conter mais do que um caractere

5. Associe os seguintes tipos aos correspondentes formatos de leitura e escrita.

[1] int	[2] %e
[2] float	[5, 6] %ld
[3] char	[2] %f
[4] short int	[1] %d
[5] long int	[4] %hd
[6] signed long int	[3] %c

6. Indique quais das seguintes afirmações são verdadeiras e quais são falsas.

- a) O tipo float reserva espaço em memória para um real com precisão simples, enquanto o tipo double reserva espaço para uma variável com precisão dupla. - **verdadeira**
- b) O tipo char pode ter os prefixos long e short. - **falsa**
- c) O tipo char pode ter os prefixos signed e unsigned. - **verdadeira**
- d) Uma variável declarada como unsigned pode comportar valores superiores a uma outra que seja declarada como signed. - **verdadeira**
- e) Uma variável do tipo char pode armazenar caracteres individuais ou conjuntos de caracteres também denominados por string. - **falsa**

- f) Uma variável do tipo char pode armazenar vários caracteres, desde que todos eles sejam caracteres ASCII. - **falsa**
- g) Uma variável do tipo char pode armazenar vários caracteres, desde que sejam caracteres especiais. - **falsa**
- h) O operador módulo (%) não pode ser utilizado em reais. - **verdadeira**

7. Identifique os erros de compilação que seriam detectados nos seguintes programas:

7.1:

```
/*  
 * Copyright: Asneira Suprema Software!!!  
 */  
  
#include  
main()  
{  
    int x, y, x;  
}
```

Em C, uma variável só pode ser declarada uma única vez dentro de um mesmo escopo. Uma redeclaração, como ocorre com o identificador 'x' no programa, ocasiona um erro de compilação.

7.2:

```
/*  
 * Copyright: Asneira Suprema Software!!!  
 */  
  
#include  
main()  
{  
    int x, y;  
    float int = 5.23;
```

```
printf("%f", int);  
}
```

Na declaração de uma variável, após especificar o tipo de dado, deve-se definir o seu identificador. Este deve seguir certas regras de nomeação; uma delas estabelece que não se podem utilizar palavras reservadas da linguagem (como 'int', no exemplo dado). O programa não será compilado devido a essa violação sintática.

7.3:

```
/*  
 * Copyright: Asneira Suprema Software!!!  
 */  
  
#include  
main()  
{  
    int x=y=z=0;  
    printf("%d %d %d \n", x, y, z);  
}
```

C não permite atribuições encadeadas diretamente na declaração de variáveis. Isso ocorre devido à forma como a operação de atribuição é processada: o compilador avalia primeiro o operando da direita. No código de exemplo, o erro acontece porque o compilador tenta atribuir valores a 'y' e 'z' antes que eles tenham sido formalmente declarados, impedindo a compilação.

8. Embora os programas que se seguem não tenham erros de compilação, identifique as causas de seu possível mau funcionamento.

8.1:

```
/*
```

```

* Copyright: Asneira Suprema Software!!!
*/

#include
main()
{
    int n;
    scanf("Introduza um N° %d", &n);
    printf("O n° = %d\n", n);
}

```

Se um texto for passado dentro da string de formato da função `scanf`, ele não será exibido na tela. Em vez disso, esse texto torna-se um padrão obrigatório que a função esperará encontrar na entrada de dados. Caso a entrada fornecida pelo usuário não seja exatamente igual ao padrão definido, a leitura não será concluída corretamente. No exemplo de código fornecido, um literal de string foi passado na chamada da função. Se o usuário não digitar exatamente esse literal antes do número, o `scanf` interromperá a leitura, falhando na atribuição do valor à variável `n`. Como a variável não foi inicializada e a atribuição falhou, ao tentar imprimir seu valor, o resultado exibido na tela será um "lixo de memória".

8.2:

```

/*
* Copyright: Asneira Suprema Software!!!
*/

#include
main()
{
    int n;
    printf("Introduza um N°");
    scanf("%d\n", &n);
    printf("O n° = %d\n", n);
}

```

No código acima, o caractere de nova linha (`\n`) passado na string de formato do `scanf` (logo após o especificador de inteiro `%d`) não terá o mesmo efeito que teria na função `printf`. Em vez de gerar um efeito visual de quebra de linha durante a execução, ele instrui o `scanf` a ignorar qualquer caractere que represente "espaço em branco" (como espaços, tabulações e quebras de linha). Assim, quando o usuário digitar o valor para a variável e pressionar `Enter`, essa tecla não finalizará a leitura, pois será ignorada pela função. Para interromper esse ciclo, será necessário que o usuário insira qualquer outro caractere (não branco) e pressione `Enter` novamente, permitindo que o programa retorne à execução normal.

8.3:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

#include
main()
{
    int n;
    printf("Introduza um N°");
    scanf("%f", &n);
    printf("O n° = %f\n", n);
}
```

No código de exemplo, existe uma incompatibilidade de tipos tanto na função `scanf` quanto na função `printf`. Está sendo feita a leitura de um dado do tipo `float`, mas a variável associada a essa atribuição está declarada como `int`. Como existe uma diferença na disposição dos bits (ou em como esses bits são armazenados), o valor real que será informado não será atribuído corretamente.

9. Escreva um programa que solicite ao usuário uma determinada data e a mostre em seguida no formato dd/mm/aaaa

[Ver código-fonte \(exe09.c\)](#)

10. Escreva um programa que solicite ao usuário uma determinada data no formato aaaa-mm-dd e a mostre em seguida no formato dd/mm/aaaa

[Ver código-fonte \(exe10.c\)](#)